CASPER

CS683: Advanced Computer Architecture-2024

L9-The world of Maya!!

https://www.cse.iitb.ac.in/~biswa/courses.html

https://www.cse.iitb.ac.in/~biswa/

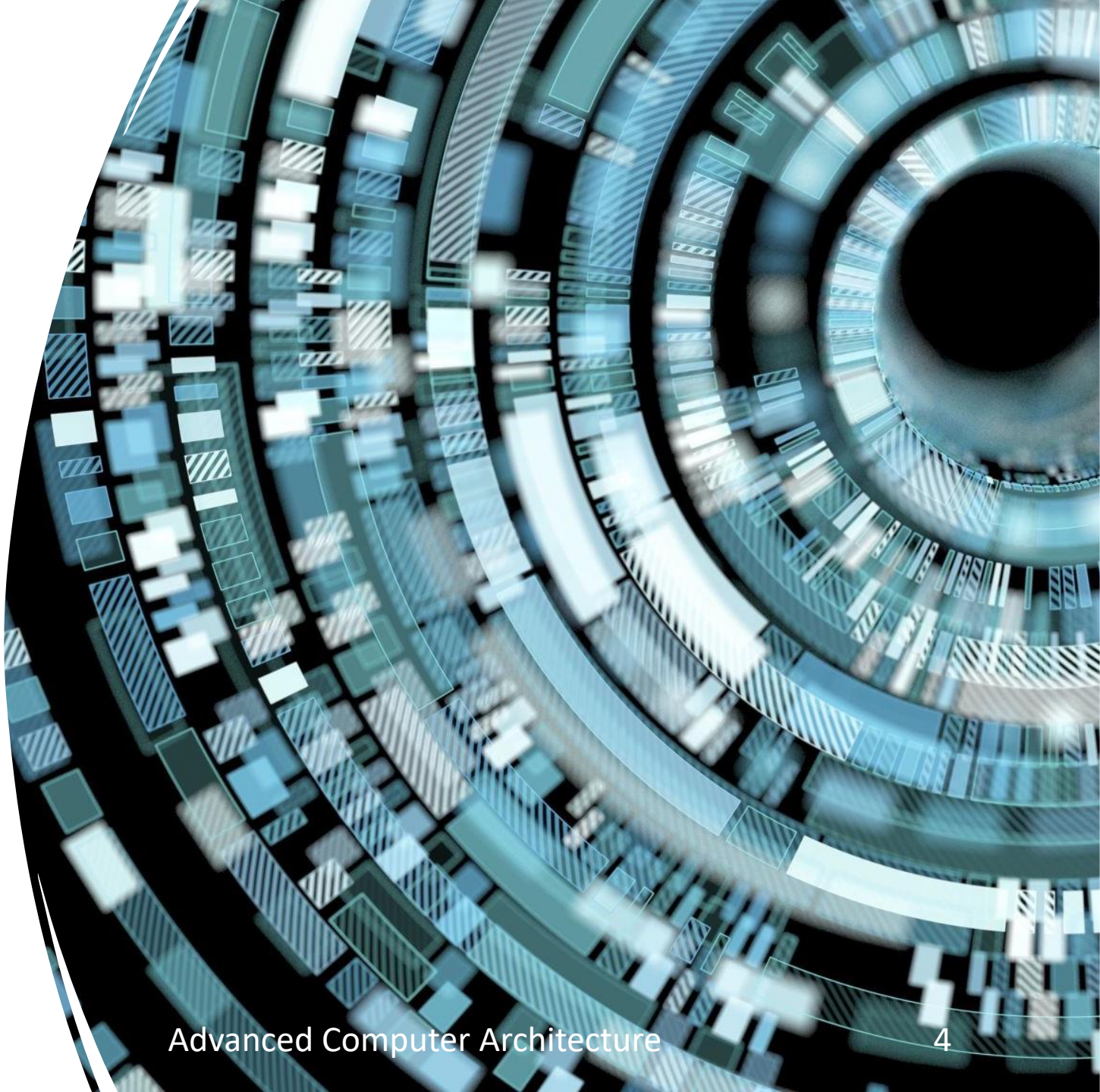
Viva slot: Fill the form by 5 PM, Run run run !!

Phones (smart/non-smart) on silence plz, Thanks

OS

- What is it?
- OS: The world of Maya

Virtual world: Virtual memory, Process (virtual CPUs)



Operating System and Architecture: Bandish 101

Case 1: Programmer wants to run 100 things

CPU says I am alone ☹

OS says I can create an illusion of multiple CPUs ☺

Case 2: Programmer wants 100s of GBs of data

Memory says I am just 10 GB ☹

OS says, never mind, I can create an illusion of TBs ☺

Operating System and Architecture: Bandish 101

Case 3: Programmer wants protection/security of data
OS says I can do it but need your support ☹
CPU says sure ☺
Case 4: Programmer wants parallelism
OS says, why not ☺ but the cost of parallelism ☹
CPU says I will take care ☺

From a program to a process

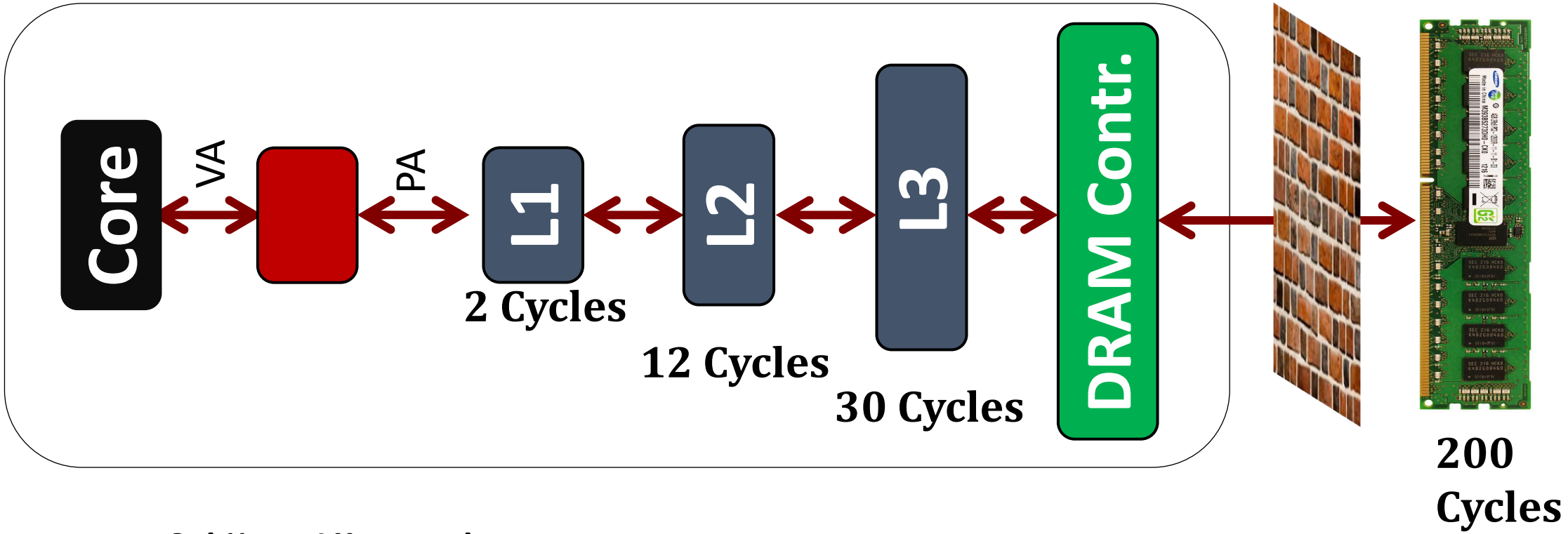
Process: A program that is alive and not-dead (running, waiting ..) ☺

OS creates, manages, schedules them

Allocates memory and initialize CPU state (PC) to kickstart

OS can run multiple processes concurrently even on a single core

Virtual World



Printf (“%d”, &a);

Virtual address

Virtual Memory

App. 1

Virtual address space

Printf (“%d”, &a);

App. 2

Virtual address space

Printf (“%d”, &a);

A bit of detour towards OS: Paging

Memory space divided into pages.
Typical page size: 4KB
Huge page: 2MB, 1GB pages

A software table that stores the paging information: Page table

An entry in the page table is known as page-table entry (PTE)

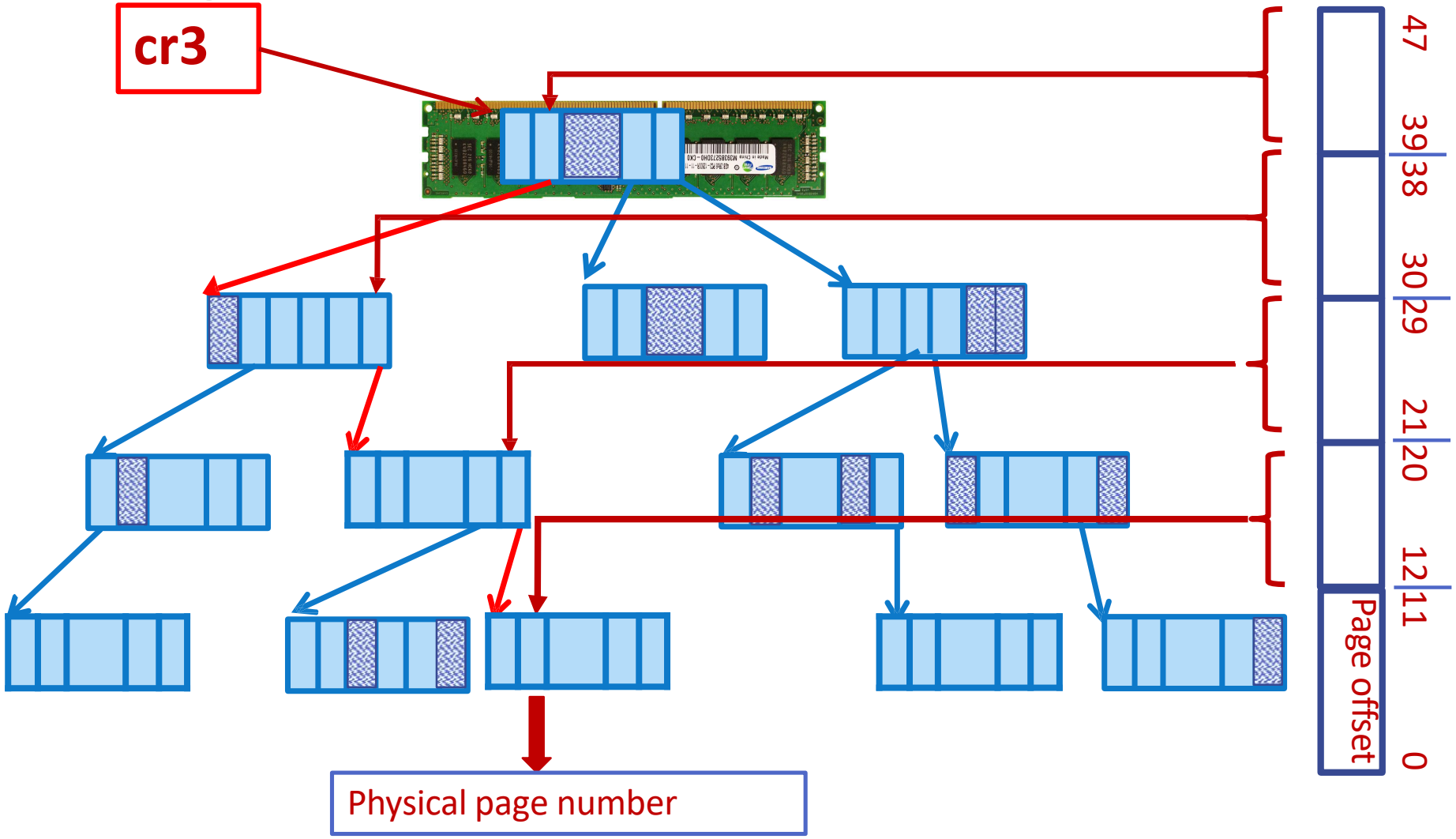
Per process page table (stored in memory)

Multiple levels to save space

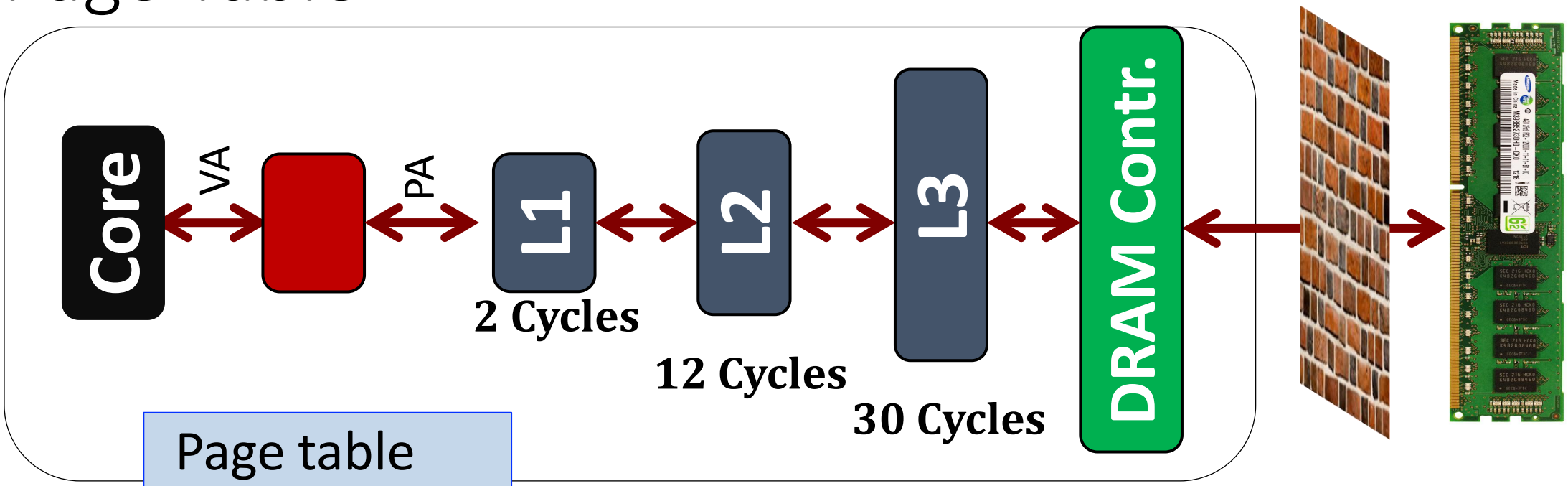
Please note that the page table does not store both virtual and physical pages. It is indexed by virtual page and physical page is stored in the page table entry.

Virtual page	Physical page

Page Table Walk



Page Table



Page table

VP0 → PP8
VP1 → PP3
VP2 → PP2

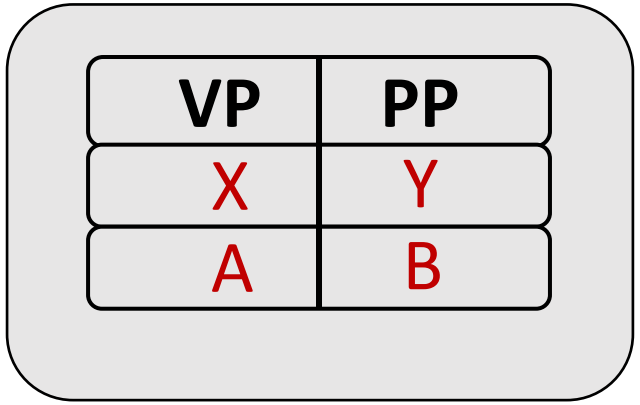
Translation Steps

H/W: for each mem reference:

(cheap)		extract **VPN** (virt page num) from **VA** (virt addr)
(cheap)		calculate addr of **PTE** (page table entry)
(expensive)		read **PTE** from memory
(cheap)		extract **PFN** (page frame num)
(cheap)		build **PA** (phys addr)
(expensive)		read contents of **PA** from memory into register

Which Steps are expensive?

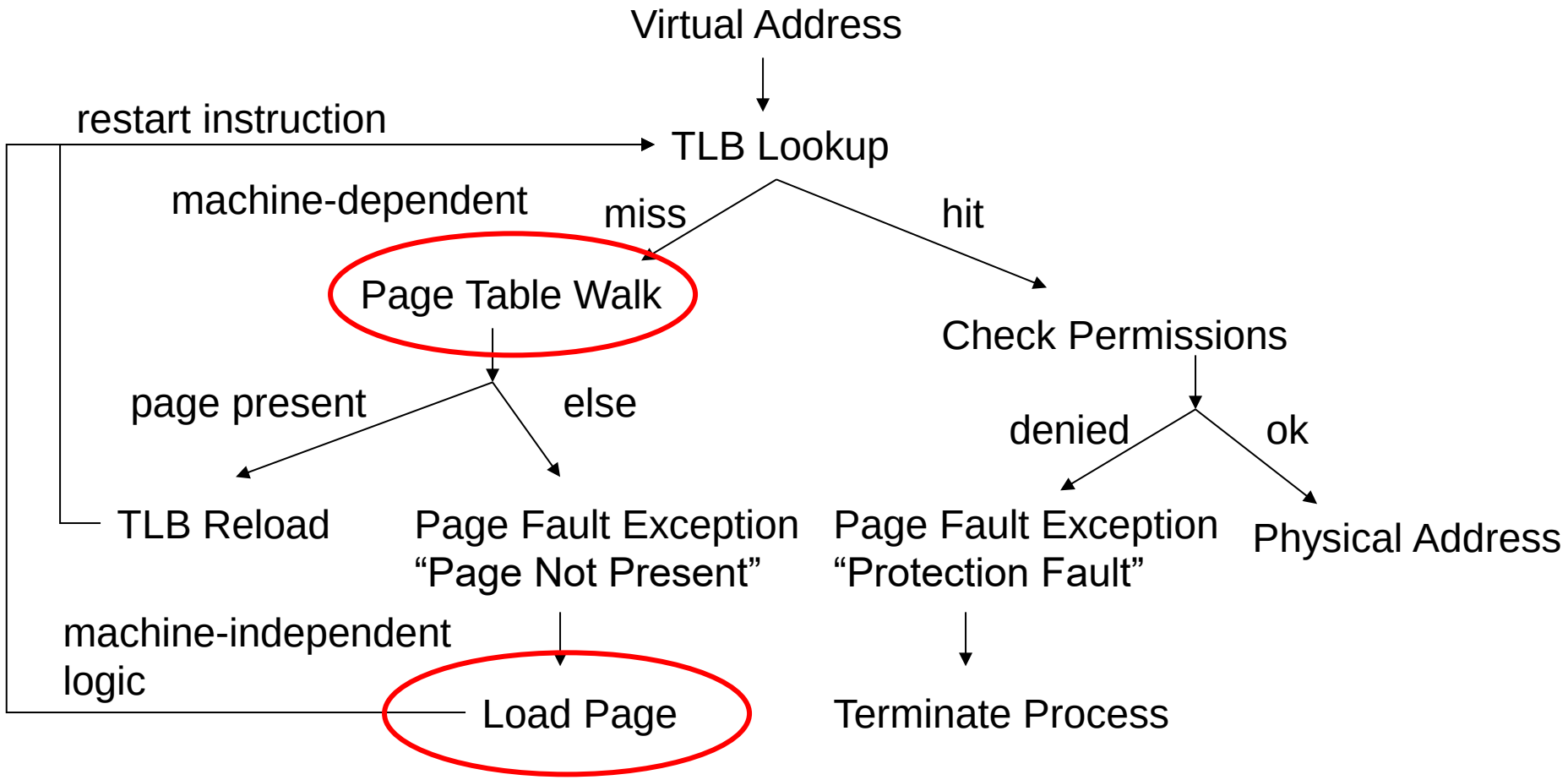
Can We Cache Translations too?



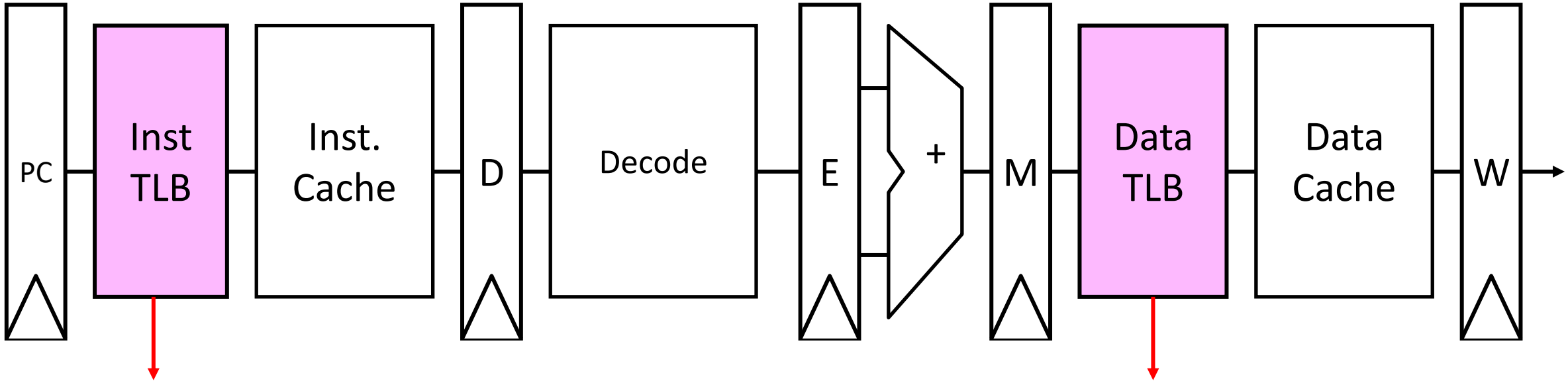
Translation Look-aside Buffers (TLBs)

Large pages
Small pages

TLBs and events of interest



The Processor Pipeline with the TLBs



Let’s Dig Deep into TLBs

TLB Entry

- TLB is a cache of page table
- Each TLB entry should cache all the information in a PTE
- It also needs to store the VPN as a tag
  - To be used when the hardware searches TLB for a particular VPN

TLB Entry

Tag (Virtual Page Number)	**Page Table Entry** (PFN, Permission Bits, Other flags)

Calculate miss rate of TLB for data: # TLB misses / # TLB lookups

```
int sum = 0;
for (i = 0; i < 2048; i++) {
        sum += a[i];
}
```

TLB lookups?

= number of accesses to a = 2048

TLB misses?

= number of unique pages accessed
= 2048 / (elements of 'a' per 4K page)
= 2K / (4K / sizeof(int)) = 2K / 1K
= 2

Would hit rate get better or worse with smaller pages?
Answer: Worse

Miss rate?

2/2048 = 0.1%

Hit rate? (1 – miss rate) 99.9%

TLB & Workload Access Patterns

- Sequential array accesses almost always hit in TLB
  - Very fast!
- What access pattern will be slow?
  - Highly random, with no repeat accesses

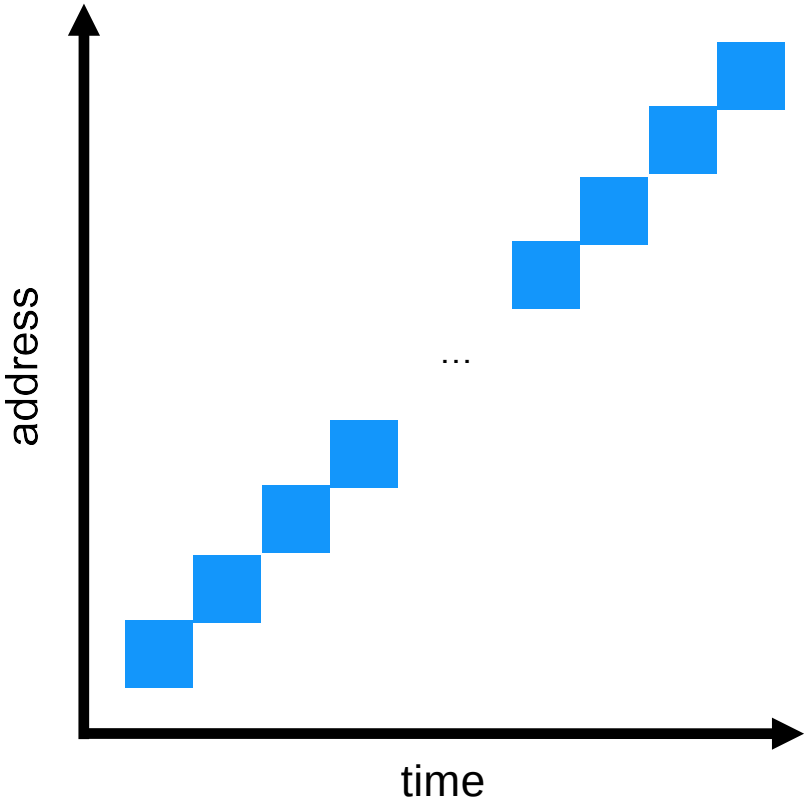
Workload A

```
int sum = 0;

for (i=0; i<2000; i++) {
       sum += a[i];

}
```

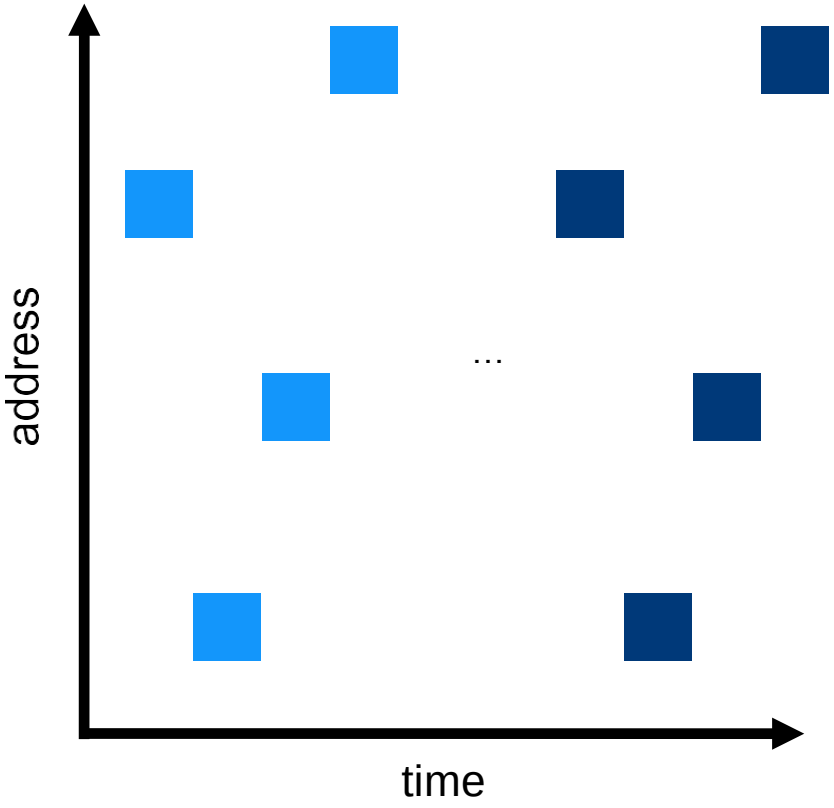
Workload B

```
int sum = 0;
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
srand(1234);
for (i=0; i<1000; i++) {
   sum += a[rand() % N];
}
```

Sequential Accesses
(Good for TLB)



Random Accesses
(Bad for TLB)



TLB Performance

- How can system improve TLB performance (hit rate) given fixed number of TLB entries?

- Increase page size
  - Fewer unique page translations needed to access same amount of memory

- What is the drawback of large pages?
  - Increased internal fragmentation
  - Tradeoffs, tradeoffs

- Most processors support multiple different page sizes
  - In 32-bit x86, 4KB and 2MB
  - In 64-bit x86, 4KB, 2MB and 1GB
  - Programmer (user) makes the choice since they know their program

- **TLB Reach**: Number of TLB entries * Page Size

TLBs and Context Switches

- What happens if a process uses cached TLB entries from another process?

Solutions?

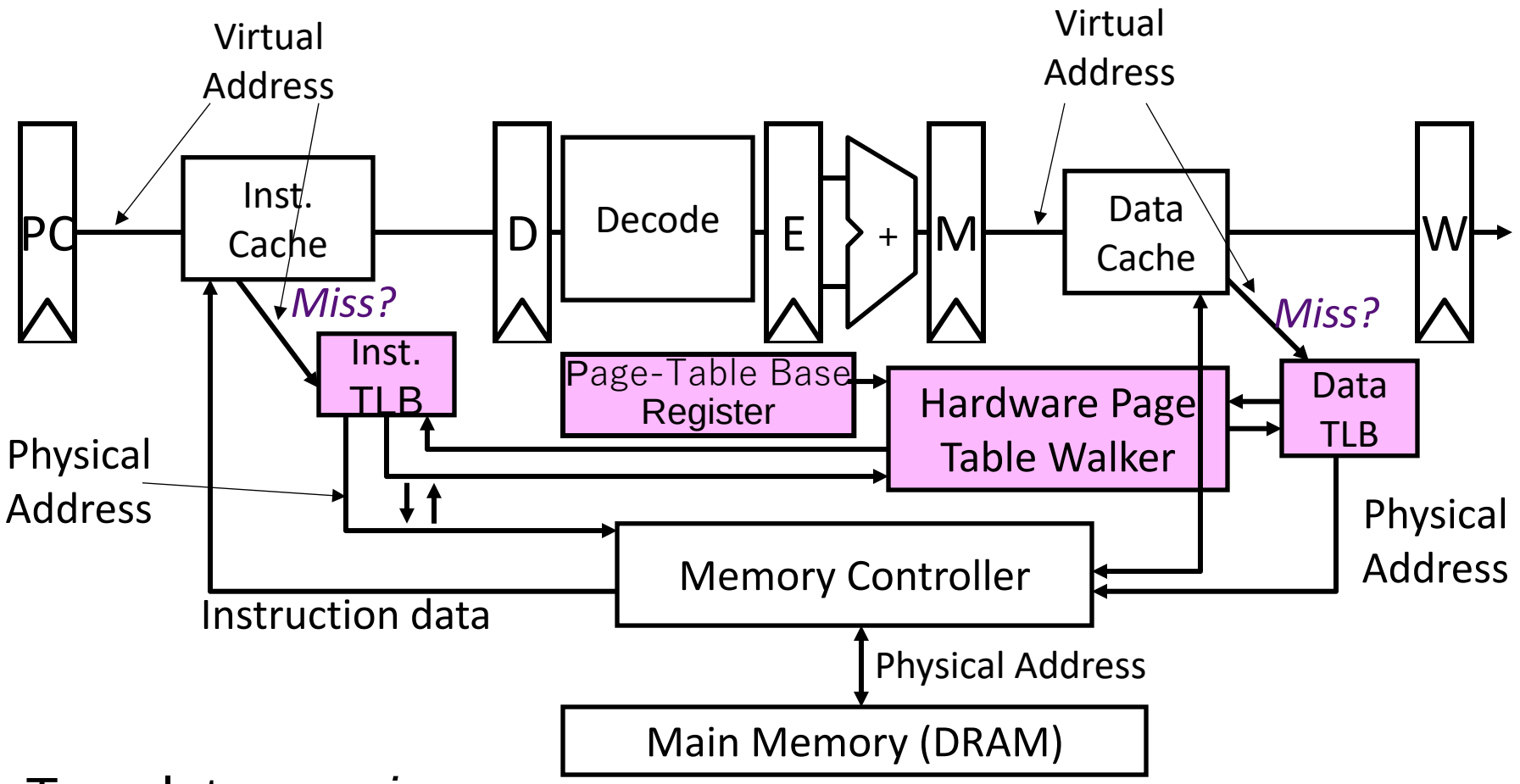
1) Flush TLB on each switch
   - Could be costly; lose all recently cached translations

2) Track which entries are for which process
   - Address Space IDentifier (ASID)
   - When loading a TLB entry, tag it with the ASID of the current process (in addition to the VPN)
   - When reading TLB, check both VPN and ASID

TLB Performance

- Context switches are expensive
- Even with ASID, other processes “pollute” TLB
  - Discard process A’s TLB entries for process B’s entries
- Architectures often have multiple TLBs and multi-level TLBs
  - Level-1: 1 TLB for data, 1 TLB for instructions (smaller, say 64 entries; fast)
  - Level 2: combined or separate inst or data TLBs (larger, say 512 entries; slower)
  - On a Level-1 TLB miss, first check Level-2 TLB before checking the page table
  - MMU caches too: Stores mid-level translations

A bit Deeper



Translate on *miss*

Please read the Book:Three easy pieces